

M4.3Live-v2 — rapport de conception

Libérer le sens du prêt : décisions d'architecture, preuves de neutralité, et statut de la parité

Programme M4.3Live-v2 — dossier `m4_3live_v2_credit_soc`

22 août 2026

Résumé

Ce document justifie les décisions d'architecture de M4.3Live-v2 et donne, pour chacune, la preuve ou la mesure qui la soutient. Le changement central est que **le sens du prêt n'est plus imposé** : dans une paire, ce n'est plus la plus riche qui prête, c'est l'optimum de production jointe qui décide, et son signe est libre. Trois changements l'accompagnent : la purge des clefs mortes du carnet de prêts, qui ramène le coût d'un run de quadratique à linéaire en horizon ; la suppression d'une borne de transfert qui n'a jamais mordu ; et un ordre de phases configurable.

Deux exigences ont guidé chaque choix. La première est que **le moteur v1 reste gelé** : v2 en est une copie, jamais un import, parce que v1 a produit 203 runs enregistrés et deux rapports publiés. La seconde est que **la parité bit à bit avec M4.3 en régime homogène soit reconduite**, ce qu'elle est : 8000 pas $\times$ 26 colonnes, écart maximal nul, après chaque lot. S'y ajoute une garantie plus forte pour la campagne : sous `loan_direction="richest_lends"`, le moteur v2 reproduit le moteur v1 *bit à bit y compris en régime hétérogène*, ce qui rend la comparaison « ancienne règle contre sens libre » appariée et non inter-lignée.

Table des matières

1	Le modèle, et les notations employées	2
1.1	Ce que simule M4.3Live-v2	2
1.2	Un pas de temps	2
1.3	Notations	4
1.4	L'institution de principal, et le sens du prêt	4
1.5	La tension, et le capital équivalent	5
1.6	Deux groupes de covariance	6
1.7	Les portées d'intervention	7
2	Le mandat, et ce qui en découle	7
2.1	La règle du fork	7
2.2	Ce qui a été volontairement laissé de côté	7
3	Le sens du prêt : ce qui change exactement	8
3.1	L'état de v1, et ce qu'il coûtait	8
3.2	La paire devient non ordonnée	8
3.3	Le taux d'intérêt n'a pas eu à changer	9
3.4	Le compteur contrefactuel, et sa limite	10
3.5	Les diagnostics du régime nouveau	10
4	La purge des clefs mortes du carnet	10
4.1	Le diagnostic, et le coût correctement énoncé	10
4.2	La correction, et la preuve qu'elle est neutre	10
4.3	La mesure	11
5	La borne de transfert supprimée	12

6	L'ordre des phases, en option	12
7	L'instrumentation devenue native	13
7.1	La tension	13
7.2	L'amplitude exacte	14
7.3	La prédiction naïve du levier γ	14
8	Les trois sémantiques de parité, et leur état	15
8.1	La seule différence, expliquée ligne à ligne	16
8.2	La garantie qui compte vraiment pour la campagne	16
9	Ce que la suite de tests couvre	16
10	Les décisions laissées ouvertes, et comment elles ont été tranchées	18
11	Un trou assumé : les annotations de conception de v1	18
12	Périmètre : ce qui a été retiré, et à la demande de qui	18
13	Un écart au séquençement demandé, et sa portée	19
14	Reproduire	19
A	Annexe de traçabilité	19

1 Le modèle, et les notations employées

1.1 Ce que simule M4.3Live-v2

Le modèle décrit une population d'entités économiques abstraites qui échangent une ressource unique, le *joule*, assimilée ici à un capital. Il n'y a ni monnaie, ni banque centrale, ni prix : seulement des capitaux, une fonction de production, et des prêts bilatéraux perpétuels.

Une entité i est entièrement décrite par trois nombres : son capital $K_i \geq 0$ et sa *technologie* (A_i, γ_i) , qui fixe la quantité qu'elle produit à chaque pas de temps,

$$\text{production de } i = A_i K_i^{\gamma_i}, \quad A_i > 0, \quad 0 < \gamma_i < 1. \quad (1)$$

L'exposant $\gamma_i < 1$ rend les rendements *marginiaux* décroissants : un joule supplémentaire rapporte d'autant moins que le capital est déjà grand. La production, elle, continue de croître avec le capital — doubler le capital augmente la production, mais de moins du double. C'est cette concavité qui crée un intérêt à déplacer du capital d'une entité vers une autre : c'est le fondement du marché du crédit du modèle.

1.2 Un pas de temps

Chaque pas déroule huit phases. Leur ordre est fixe, à une exception près, signalée ci-dessous.

1. **Interventions** — les changements de paramètres demandés en direct sont appliqués (voir §1.7 pour leurs portées).
2. **Naissances** — un nombre Poisson(λ) d'entités neuves apparaissent, chacune dotée du capital de naissance K_0 .
3. **Choc** — chaque capital est multiplié par e^ξ avec $\xi \sim \mathcal{N}(-\sigma^2/2, \sigma^2)$, un choc multiplicatif d'espérance 1 (il ne crée ni ne détruit de richesse en moyenne). *Ordre de grandeur*. L'écart typique se lit sur $\mathbb{E}|\xi| = \sigma\sqrt{2/\pi}$ à la correction de moyenne près ($-\sigma^2/2$, négligeable ici) : à $\sigma = 0,01$, la valeur employée dans tout ce programme, $\mathbb{E}|\xi| \approx 8,0 \times 10^{-3}$, donc un pas multiplie un capital par $1 \pm 0,8\%$ en moyenne.

4. **Production** — chaque entité produit selon (1) et *ajoute intégralement* sa production à son propre capital.
5. **Service des intérêts** — chaque débitrice doit $dû = \sum_k q_k r_k$, somme sur ses contrats du principal q_k fois le taux r_k . Si son capital ne suffit pas, elle paie ce qu'elle peut au prorata, tombe à zéro et est marquée en *défaut de liquidité*.
6. **Dépréciation** — chaque capital est multiplié par $1 - \delta$.
7. **Marché du crédit** — $\lfloor \eta(N) \rfloor$ tirages de paires, où N est le nombre d'entités éligibles et $\eta_{\rho,\beta}(N) = \rho N_{\text{ref}}(N/N_{\text{ref}})^\beta$ (la référence $\rho = 1, \beta = 1$ donne simplement $\eta(N) = N$). Chaque paire tirée traite ou ne traite pas ; si elle traite, un contrat transfère un principal q et fixe un taux r , tous deux gelés à vie.
8. **Faillites en cascade** — toute entité dont la valeur nette $K + \text{créances} - \text{dettes}$ devient négative, ou qui est en défaut de liquidité, meurt : ses dettes sont annulées (perte sèche pour ses créancières, qui peuvent tomber à leur tour) et son capital est détruit. On itère jusqu'à point fixe.

L'ordre des phases 5 et 6 est un paramètre. Le champ `phase_order` vaut "v1" par défaut — l'ordre ci-dessus, celui de M4.3 et de M4.3Live — ou `deprec_first`, qui déprécie avant de servir les intérêts. Sous cette seconde valeur, le capital disponible au moment de payer est réduit d'un facteur $1 - \delta$: bascule en défaut de liquidité toute débitrice dont le capital K vérifie $(1 - \delta)K < dû \leq K$. Le rapport de résultats mesure cet effet contre cette prédiction.

Un identifiant d'entité n'est jamais réutilisé ; une entité morte le reste.

1.3 Notations

Symbole	Nom	Définition
K_i	capital	Ressource détenue par l'entité i .
A_i	coefficient d'extraction	Facteur multiplicatif de la production (1). Levier primaire de ce programme.
γ_i	exposant d'extraction	Exposant de la production ; il fixe le rendement d'échelle et rend les rendements <i>marginiaux</i> décroissants ($0 < \gamma < 1$). Levier secondaire.
(A_i, γ_i)	technologie	Couple fixé à la naissance, jamais modifié sauf par une intervention. Reçoit un identifiant entier interne.
λ	taux de naissance	Espérance du nombre d'entités créées par pas.
K_0	capital de naissance	Capital dont une entité neuve est dotée.
δ	dépréciation	Fraction du capital perdue à chaque pas.
σ	amplitude du choc	Écart-type du choc multiplicatif log-normal.
ρ	taux d'appariement	Nombre de paires tirées par entité et par pas (à $\beta = 1$, le nombre de rondes est $\lfloor \rho N \rfloor$).
N	taille du bassin	Nombre d'entités éligibles au marché à ce pas.
a, b	membres de la paire	Paire non ordonnée : a et b ne sont ni prêteuse ni emprunteuse a priori (§1.4).
K_a, K_b	capitaux de la paire	Par convention de calcul, $K_a \leq K_b$.
C	somme conservée	$C = K_a + K_b$; le transfert la laisse inchangée.
q	principal	Montant transféré par le contrat, $q = \delta^* $.
r	taux	Service versé par pas, perpétuel : la débitrice doit qr à chaque pas, indéfiniment.
δ^*	transfert optimal	Transfert qui maximise la production jointe de la paire, de signe libre (éq. 2).
λ^*	part optimale	Part de C revenant à a à l'optimum ; $h(C) = C\lambda^*$ est son capital optimal.
K_{aut}^*	échelle autarcique	$[A(1-\delta)/\delta]^{1/(1-\gamma)}$: capital auquel production et dépréciation s'équilibrent <i>sans</i> marché ni dette.
K_{eq}	capital équivalent	$(\Pi/(nA))^{1/\gamma}$: capital qui reproduirait la production observée Π si les n entités productrices étaient toutes identiques (§1.5).
T	tension	$T = K_{\text{aut}}^*/K_{\text{eq}}$ (§1.5).
G	Gini du capital	$\mathbb{E} X - Y /(2\mathbb{E}[X])$ pour deux capitaux X, Y tirés indépendamment dans la population.
prod_tot	production agrégée	Somme des productions du pas. Variable d'intérêt.
K_tot, pop	agréats	Capital total et effectif vivant.
deaths	morts	Entités mortes au pas (racines + victimes de cascade).
defaults	défauts	Entités en défaut de liquidité au pas.
loan_volume	volume prêté	Somme des principaux transférés au pas.
mkt_rounds	tirages	Nombre de paires tirées au pas.
mkt_blocked_dir	refus de sens	Sous la règle v1, paires refusées parce que $\delta^* \leq 0$. Sous le sens libre, compteur contrefactuel (§1.4).
mkt_reversed	prêts inversés	Prêts conclus dans le sens que la règle v1 interdisait, et mkt_volume_rev leur volume.
K_share_creditors	—	Part du capital total détenue par les entités en position nette créancière.
corr_marg_net	—	Corrélation de Pearson, sur les vivantes, entre le rendement marginal $\gamma_i A_i K_i^{\gamma_i - 1}$ et la position nette créances _{i} – dettes _{i} .
defaults_window	fenêtre de bascule	Nombre de débitrices dont le statut changerait si l'on échangeait les phases 5 et 6 de §1.2.

1.4 L'institution de principal, et le sens du prêt

Là où M4.3 transférait la moitié de l'écart de capital, cette lignée transfère le montant qui maximise la production *jointe* de la paire immédiatement après l'échange. La paire $\{a, b\}$ étant **non ordonnée**,

$$\delta^* = \arg \max_{-K_a \leq \delta \leq K_b} A_a (K_a + \delta)^{\gamma_a} + A_b (K_b - \delta)^{\gamma_b} = h(C) - K_a, \quad h(C) = C\lambda^*. \quad (2)$$

L'objectif est strictement concave en δ et sa dérivée diverge aux deux bords : l'optimum est intérieur et unique.

Le signe de δ^* est libre, et c'est le changement central de v2. Si $\delta^* > 0$, a reçoit δ^* de b ; si $\delta^* < 0$, c'est b qui reçoit $|\delta^*|$ de a . Celle qui cède détient la créance, celle qui reçoit porte la dette, *quelle que soit laquelle est la plus riche*. Dans M4.3 et dans M4.3Live, la plus riche prêtait toujours, et une paire dont l'optimum demandait l'inverse était refusée. Le paramètre `loan_direction` permet de rejouer cette ancienne règle ("**richest_lends**") dans le même moteur, ce qui rend la comparaison appariée.

Trois conséquences importent pour lire les deux rapports.

1. Quand les deux entités partagent la *même* technologie, $\lambda^* = 1/2$ et $\delta^* = (K_b - K_a)/2 \geq 0$ exactement. Les deux règles de sens coïncident alors, et pas seulement approximativement : les trajectoires sont identiques *bit à bit*.
2. Dès que les technologies diffèrent, $\lambda^* \neq 1/2$ et le partage est pondéré en faveur de celle dont le *rendement marginal est le plus élevé au capital considéré*. Il n'existe pas de « meilleure » technologie dans l'absolu : à exposants inégaux, l'ordre des rendements marginaux $\gamma AK^{\gamma-1}$ s'inverse avec le capital.
3. $\delta^* \leq 0$ exactement quand $K_a/C \geq \lambda^*$, c'est-à-dire quand la moins riche est déjà au-delà de sa part optimale. Sous le sens libre, cette paire traite *quand même*, en sens inverse ; le compteur `mkt_blocked_dir` devient alors **contrefactuel** : il compte les paires que la règle v1 aurait refusées *dans l'état où v2 se trouve à cette ronde*. Ce n'est pas le compteur d'une trajectoire v1, puisque les deux trajectoires divergent dès le premier prêt inversé.

Le taux, et pourquoi il n'a pas eu à changer. Le taux d'un contrat est la moyenne géométrique des rendements marginaux des deux entités, évalués à leur capital d'*avant* l'échange :

$$r = \sqrt{m_a m_b}, \quad m_i = \gamma_i A_i K_i^{\gamma_i-1}. \quad (3)$$

Cette expression *ne contient ni rôle, ni comparaison de capitaux* : échanger les deux côtés laisse r inchangé, et l'égalité est exacte au dernier bit puisque la seule opération qui les mêle est un produit. Libérer le sens du prêt ne change donc rien à la règle de taux — y compris quand celle qui prête est la plus fragile des deux.

Surplus coopératif. Le **surplus coopératif** d'une paire est le gain de production *jointe* que l'échange produit, à capital total inchangé. En notant R l'entité qui reçoit et D celle qui cède,

$$S = \underbrace{A_R(K_R + q)^{\gamma_R} + A_D(K_D - q)^{\gamma_D}}_{\text{après le transfert}} - \underbrace{A_R K_R^{\gamma_R} + A_D K_D^{\gamma_D}}_{\text{avant}} \geq 0, \quad (4)$$

avec $q = |\delta^*|$. La positivité est immédiate : δ^* maximise le premier terme et $\delta = 0$ est admissible. S est nul exactement quand la paire est déjà à l'optimum. C'est un *flux de production par pas*, homogène à `prod_tot`, et non un stock de capital : le marché ne crée aucun joule, il déplace du capital vers l'endroit où il produit davantage.

1.5 La tension, et le capital équivalent

Une entité isolée — sans marché ni dette — voit son capital devenir $(K + AK^\gamma)(1 - \delta)$ à chaque pas. Le point fixe de cette récurrence est l'**échelle autarcique**

$$K_{\text{aut}}^* = \left[\frac{A(1 - \delta)}{\delta} \right]^{1/(1-\gamma)}. \quad (5)$$

C'est une échelle purement paramétrique : elle ne dépend ni de la population, ni du crédit, ni de l'histoire du run. En face, l'échelle à laquelle le système tourne *réellement* est définie par la production. Le **capital équivalent** K_{eq} est le capital qu'auraient toutes les entités d'un groupe

technologique si, à production agrégée Π et effectif producteur n identiques, elles étaient toutes au même capital :

$$\Pi = n A K_{\text{eq}}^\gamma \quad \Longleftrightarrow \quad K_{\text{eq}} = \left(\frac{\Pi}{n A} \right)^{1/\gamma}. \quad (6)$$

La **tension** est le rapport $T = K_{\text{aut}}^*/K_{\text{eq}}$. $T = 1$: le système tourne à son échelle autarcique. $T > 1$: il tourne *en dessous*, le capital y étant maintenu bas par tout ce que l'autarcie ignore — service des intérêts, mortalité, renouvellement par des naissances petites.

Trois conventions, énoncées parce qu'elles ne sont pas neutres. (i) K_{aut}^* ignore le service des intérêts : c'est sa définition, et c'est ce qui fait de T la mesure de tout ce qui n'est pas la technologie. (ii) K_{eq} n'est pas le capital moyen : K^γ étant concave, l'inégalité de Jensen donne $K_{\text{eq}} \leq \bar{K}$, avec égalité seulement si tous les capitaux sont égaux ; le rapport $K_{\text{eq}}/\bar{K}$ est donc rapporté à côté de T comme mesure d'inégalité interne. (iii) Le calcul est fait par groupe technologique, seul niveau où A et γ sont définis ; l'agrégat d'un run à plusieurs technologies est la moyenne des tensions pondérée par la production.

En v2, ces colonnes sont natives : l'effectif producteur n et le capital $\bar{K}$ sont enregistrés à l'instant de produire, avant les morts du pas. Le rapport de Jensen est donc exact, ses deux termes étant pris au même instant.

1.6 Deux groupes de covariance

Le modèle possède deux symétries exactes ou quasi exactes, qui réduisent le nombre de paramètres réellement libres. Elles sont énoncées ici ensemble, parce qu'elles sont deux faces du même objet : le modèle n'a pas d'échelle de capital propre, et pas d'unité de temps propre.

Covariance d'échelle (capital). Si l'on remplace A par A' et K_0 par $c K_0$ avec

$$c = (A'/A)^{1/(1-\gamma)}, \quad (7)$$

on obtient *la même simulation à l'échelle c près*. Chaque phase du pas est homogène de degré 1 en capital — la production parce que $A'(cK)^\gamma = c A K^\gamma$ dès que $A'c^\gamma = cA$, c'est-à-dire dès que $c^{1-\gamma} = A'/A$; les intérêts, la dépréciation et les transferts parce qu'ils sont linéaires — et le taux marginal $\gamma A K^{\gamma-1}$ est *sans dimension*, donc invariant. Population, nombre de contrats et morts sont rigoureusement identiques ; tous les capitaux sont multipliés par c .

Covariance de pas de temps. Si l'on comprime s pas anciens en un pas nouveau, les substitutions sont

$$\lambda \rightarrow s\lambda, \quad \delta \rightarrow 1 - (1 - \delta)^s, \quad \sigma \rightarrow \sigma\sqrt{s}, \quad A \rightarrow sA, \quad \rho \rightarrow s\rho. \quad (8)$$

Les trois premières composent *exactement* : la somme de s Poisson d'intensité λ est un Poisson d'intensité $s\lambda$, la composition de s dépréciations est $1 - (1 - \delta)^s$, et la somme de s log-normales d'écart-type σ (dérive comprise) est une log-normale d'écart-type $\sigma\sqrt{s}$. Les deux dernières sont les *flux par pas*, que l'énoncé initial de cette covariance oubliait.

Une classification qui évite un contresens. Ces deux groupes séparent les paramètres en deux familles :

Grandeurs <i>par pas</i> (elles se recalent avec l'unité de temps)	Grandeurs <i>de forme</i> (elles n'en dépendent pas)
$\delta, \sigma, \lambda, A, \rho$	γ, K_0 , règle de transfert, règle de taux

Cette distinction évite de discuter un « effet de δ » qui ne serait qu'un changement d'unité de temps.

1.7 Les portées d'intervention

Une intervention change un paramètre *pendant* qu'une simulation tourne. Sa **portée** dit qui elle touche :

all (toutes) — rétroactif *et* prospectif : toutes les entités vivantes reçoivent la nouvelle valeur, et la valeur par défaut à la naissance change aussi. La population reste *homogène*.

new (nouvelles) — *vintage* technologique : seule la valeur par défaut à la naissance change. Les entités déjà vivantes gardent la leur pour toujours ; la population se convertit par renouvellement. Deux technologies coexistent donc, et c'est la seule portée globale qui produise des paires mixtes.

fraction (une fraction φ) — une fraction φ des entités vivantes est tirée au sort une fois, et elle seule est modifiée. **Cette portée est hors du périmètre de v2**, à la demande de l'utilisateur ; le moteur la conserve, aucune campagne de ce rapport ne l'emploie.

Certaines combinaisons n'ont pas de sens et sont refusées explicitement : K_0 n'existe qu'au moment de la naissance, donc **all** et **new** y sont le même objet ; λ , δ , σ , ρ sont des paramètres de population et non des attributs d'entité, donc **new** y est un alias explicite de **all**.

2 Le mandat, et ce qui en découle

v2 porte un changement de modèle et une question théorique, qui ne sont pas de même rang : le changement de modèle est le livrable primaire, la question théorique est le programme qu'il rend possible. Le présent document traite du premier ; le rapport de résultats traite du second.

Le changement de modèle supprime la règle « dans chaque paire, la plus riche prête ». Son intérêt n'est pas cosmétique : il ouvre un régime que v1 ne pouvait pas produire, celui d'une entité de grand capital mais de faible rendement marginal qui cède son capital à une entité plus pauvre au rendement marginal plus élevé, et vit de l'intérêt.

2.1 La règle du fork

Le paquet `m4_3live_credit_soc/m4_3live/` n'est pas modifié, ni maintenant ni plus tard. Ce n'est pas une précaution de style : c'est le moteur qui a produit 203 runs enregistrés et deux rapports publiés, et toute modification invaliderait rétroactivement des résultats cités.

v2 **forke** ce paquet par copie dans `m4_3live_v2/`. [**Fait observé**] La seule dépendance de v2 envers v1 est en *lecture* : le test `tests/test_v1_equivalence.py` importe le moteur v1 pour le comparer, et `scripts/cost_profile.py` le fait tourner pour mesurer son coût. Aucun module de v2 ne l'importe.

Trois renommages accompagnent la copie, et chacun évite une collision réelle :

- le paquet passe de `m4_3live` à `m4_3live_v2`, pour que les deux puissent être importés dans un même processus — ce que font justement le test d'équivalence et le profil de coût ;
- les routes de l'interface passent de `/live` à `/live2`, pour que les deux lignées coexistent dans le même serveur `simulation_lab` : l'aiguillage v1 n'est pas débranché, un second est ajouté à côté (`simulation_lab/live_v2/`) ;
- les identifiants de run passent de `m4_3live__` à `m4_3live_v2__`, pour qu'aucun run v2 ne se confonde avec les 203 runs v1 déjà importés dans `simulation_lab`.

2.2 Ce qui a été volontairement laissé de côté

Quatorze scripts de campagne de v1 ne sont pas repris : ablation sur K_0 , loi d'échelle, covariance, balayage de tension, tension contre A , recalage temporel, banc du noyau, amplitude reconstruite *a posteriori*, analyse de tension, figures de conception, figures de protocole, figures de vérification, analyse de campagne, fabrication de tables. Leurs résultats sont **acquis et publiés** ; v2 les cite sans les refaire. Les garder aurait laissé dans le dépôt du code que rien n'appelle.

Un script disparaît pour une autre raison : `scripts/tension.py`, parce que son calcul devient natif (§7.1) et vit désormais dans le moteur.

3 Le sens du prêt : ce qui change exactement

3.1 L'état de v1, et ce qu'il coûtait

Dans v1, chaque paire tirée était immédiatement ordonnée par la richesse (`m4_3live/model.py:515-520`) : la plus riche devenait prêteuse, la plus pauvre emprunteuse. Le noyau calculait ensuite le transfert optimal δ^* ; s'il était négatif ou nul — c'est-à-dire si l'optimum voulait faire circuler le capital de la pauvre vers la riche — la paire était *refusée* et comptée dans `mkt_blocked_dir` (`model.py:525-530`).

[Fait observé] Ce compteur n'était pas marginal. Sur un run de 300 pas avec deux technologies coexistantes, il vaut 61 166 paires refusées ; c'est le chiffre que produit `tests/test_v1_equivalence.py`, et il est identique dans les deux moteurs, ce qui est le contrôle. Sur la campagne de v2, la part des rondes concernées atteint 23,66 % dans les deux cents pas qui suivent une intervention.

3.2 La paire devient non ordonnée

Le code de marché ne connaît plus de prêteuse ni d'emprunteuse. Il connaît une paire $\{a, b\}$ et un transfert de signe libre $\delta^* = h(C) - K_a$. Si $\delta^* > 0$, a reçoit ; sinon, b reçoit. Les rôles de *donneuse* et de *receveuse* sont ensuite dérivés, et c'est la receveuse qui porte la dette.

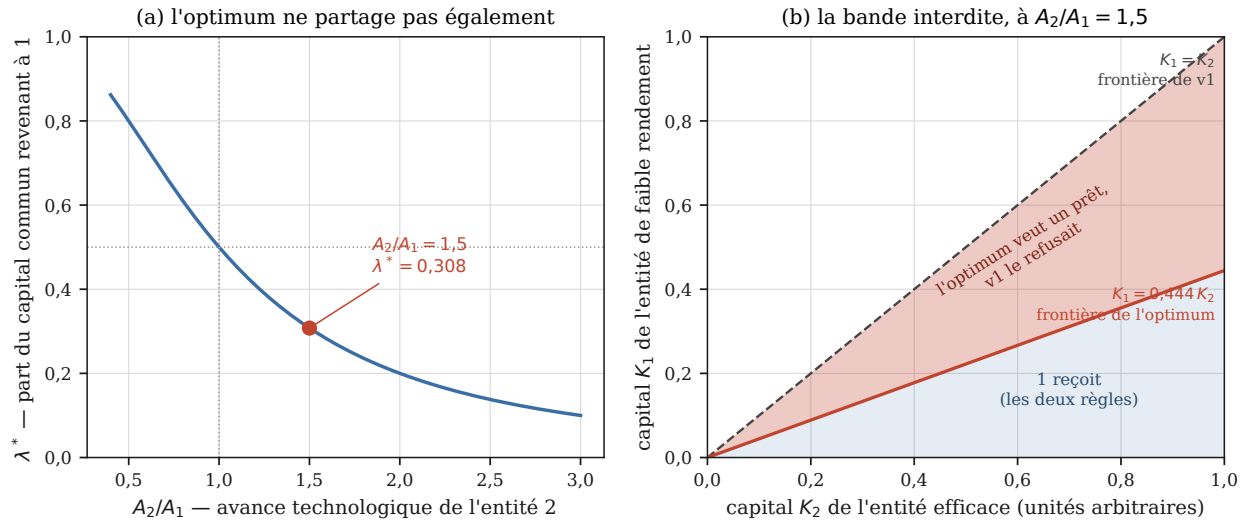


FIGURE 1 – Ce que la règle de richesse s'interdisait, avant toute mesure. À gauche, la part λ^* que l'optimum de production jointe laisse à l'entité la moins efficace, en fonction de l'avance technologique de l'autre : elle vaut 1/2 en régime homogène — où les deux règles coïncident donc exactement — et 0,308 à $A_2/A_1 = 1,5$. À droite, la conséquence dans le plan des capitaux : entre les deux droites, l'optimum veut faire circuler le capital de la moins riche vers la plus riche, et v1 refusait la paire. C'est la région que ce chantier ouvre, et sa largeur ne dépend que du rapport des technologies.

La nomenclature n'est pas cosmétique. Tant que le code dit « prêteuse », un lecteur suppose la règle de richesse, et une régression peut s'y cacher. Les variables de la boucle de marché s'appellent donc `a`, `b`, `donor` et `receiver`. `LoanBook` conserve en revanche `by_lender` et `by_borrower` : ce sont les rôles du *contrat* — qui détient la créance, qui porte la dette — et non ceux de la paire. La note de l'utilisateur qui demande l'abandon de la nomenclature vise explicitement K_ℓ, K_b , c'est-à-dire les capitaux de la paire, et c'est cette lecture qui a été retenue.

Un ordre de calcul canonique, et pourquoi il en faut un. a est, par convention, l'entité de plus petit capital de la paire, b l'autre. Ce n'est plus un rôle : c'est l'ordre dans lequel la paire est présentée au noyau. Deux raisons l'imposent.

1. **[Fait observé]** `kernel.solve(s_a, s_b, K_a, K_b)` et `kernel.solve(s_b, s_a, K_b, K_a)` sont deux *entrées différentes* de la matrice de noyaux, dont les tables d'interpolation sont compilées séparément (`kernel.py`, `PrincipalKernel._entry`). En mathématiques $h_{ab}(C) + h_{ba}(C) = C$; rien ne le garantit au dernier bit. Fixer l'ordre rend le transfert univoque.
2. **[Inférence]** Conserver l'ordre de `v1` rend la requête au noyau identique à celle de `v1`. C'est la condition pour que `richest_lends` rejoue `v1` exactement — vérifié, §8.

3.3 Le taux d'intérêt n'a pas eu à changer

Le prompt laissait ouverte la question : la règle de taux a-t-elle encore un sens quand la créancière est la plus fragile des deux ? **[Fait observé]** La réponse est que la question ne se pose pas, et on peut le lire dans la formule. Le taux est $r = \sqrt{m_a m_b}$ avec $m_i = \gamma_i A_i K_i^{\gamma_i - 1}$ (éq. 3) : elle ne contient ni rôle, ni comparaison de capitaux. Échanger les deux côtés laisse r inchangé *au dernier bit*, la seule opération qui les mêle étant un produit, dont la version flottante est commutative. C'est vérifié par une assertion d'égalité stricte, et non par une tolérance (`tests/test_loan_direction.py`).

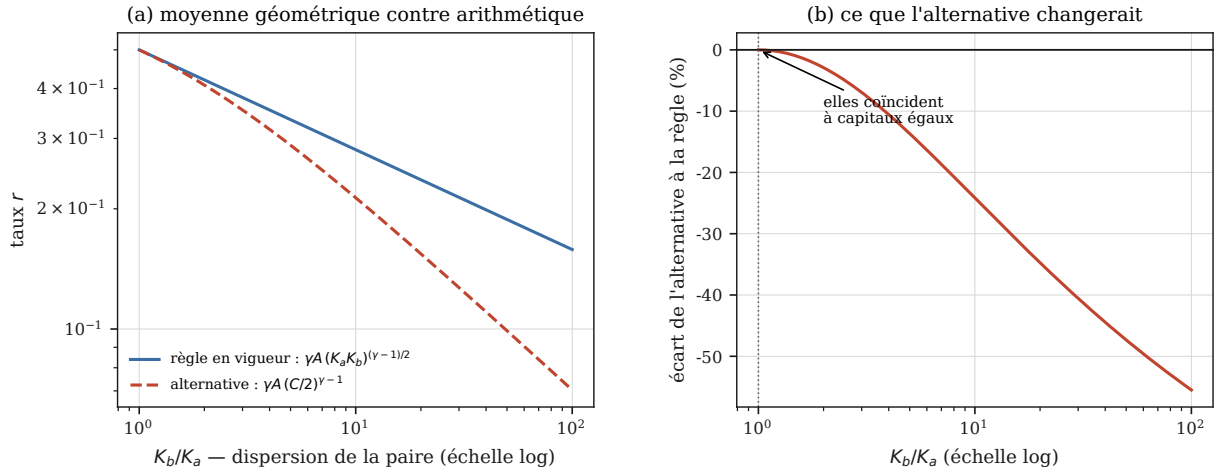


FIGURE 2 – **La règle de taux ne contient ni rôle ni comparaison de capitaux, et l'alternative n'aurait pas cette propriété.** À gauche, le taux en vigueur — moyenne *géométrique* des rendements marginaux — et le rendement marginal commun d'après l'échange, qui est une moyenne *arithmétique* des capitaux, en fonction de la dispersion de la paire. À droite, leur écart. Ce qu'il faut regarder est qu'ils ne coïncident qu'à capitaux égaux : adopter l'alternative changerait le taux de toute paire dispersée, donc la trajectoire, donc la parité — ce serait un chantier à part entière et non un ajustement.

[Incertitude] Une alternative existe et mérite d'être nommée, parce qu'elle est plus naturelle qu'il n'y paraît : à l'optimum, les deux rendements marginaux d'*après* l'échange sont égaux par construction (c'est la condition du premier ordre), et ce rendement commun est un taux candidat. Il n'est pas retenu, pour deux raisons. Il changerait l'institution au-delà du mandat du chantier, qui porte sur le *sens* et non sur le prix. Et il détruirait la parité : en régime homogène il vaut $\gamma A (C/2)^{\gamma-1}$ tandis que la règle actuelle vaut $\gamma A (K_a K_b)^{(\gamma-1)/2}$, soit la moyenne *arithmétique* des capitaux au lieu de la *géométrique*. Ce serait un chantier à part entière, mesurable comme une ablation.

3.4 Le compteur contrefactuel, et sa limite

Sous le sens libre, aucune paire n'est plus refusée pour cause de sens : `mkt_blocked_dir` tomberait à zéro par construction. Le compteur est conservé et alimenté par ce qu'il aurait valu sous l'ancienne règle. Le garde $K_b > K_a$ reproduit l'arbre de décision exact de `v1`, qui écartait les capitaux égaux *avant* de compter un refus de sens.

[**Incertitude**] Cette mesure est instantanée, pas trajectorielle. Elle compte les paires que la règle `v1` aurait refusées *dans l'état où `v2` se trouve à cette ronde*. Ce n'est pas le compteur d'une trajectoire `v1` : les deux trajectoires divergent dès le premier prêt inversé. Toute lecture de ce nombre doit le dire, et le rapport de résultats le redit à chaque emploi.

Deux compteurs nouveaux mesurent, eux, ce qui a réellement lieu : `mkt_reversed` (les prêts conclus dans le sens que `v1` interdisait) et `mkt_volume_rev` (leur volume).

3.5 Les diagnostics du régime nouveau

Le régime que ce chantier ouvre demande ses propres observables, et ils sont calculés à chaque pas sur la population vivante :

- `n_creditors` et `K_share_creditors` : l'effectif des entités en position nette créancière et la part du capital qu'elles détiennent ;
- `corr_marg_net` : la corrélation de Pearson entre le rendement marginal $\gamma_i A_i K_i^{\gamma_i-1}$ et la position nette créances_{*i*} – dettes_{*i*} ;
- `corr_K_net` : la même corrélation avec le capital. Sous la règle `v1` elle est mécaniquement positive — la plus riche prête toujours — et sa chute sous le sens libre est la signature directe du chantier.

[**Inférence**] Ces corrélations sont calculées en une passe à partir de sommes cumulées, forme numériquement moins stable que la forme centrée. C'est acceptable ici, et il faut dire pourquoi : elles n'entrent dans *aucun* calcul de la trajectoire. Ce sont des colonnes écrites dans la série, jamais relues par le moteur.

4 La purge des clefs mortes du carnet

4.1 Le diagnostic, et le coût correctement énoncé

`LoanBook.by_borrower` est un `defaultdict[int, set[int]]` (`m4_3live/model.py:307` dans `v1`) et `remove()` retire un prêt par `.discard()` : la clef reste, avec un ensemble vide. Une entité qui meurt voit tous ses prêts retirés et laisse donc derrière elle une entrée vide, définitive. La phase de service des intérêts itère sur *toutes* les clefs et fait `continue` sur les vides.

[**Fait observé**] Le nombre de clefs croît donc comme le nombre d'entités *jamais créées*, soit $\approx \lambda t$. Le coût *par pas* de la phase d'intérêts est en λt , et le coût **total** d'un run est la somme sur les T pas, soit $\lambda T^2/2$: **quadratique en horizon**.

4.2 La correction, et la preuve qu'elle est neutre

`LoanBook.forget(entity)` retire l'entité de `by_borrower`, `by_lender`, `due`, `claims` et `debts`. Elle est appelée dans `_fail_one`, juste après `population.kill`, et **nulle part ailleurs**. La méthode lève une exception si l'entité porte encore un contrat : l'invariant est vérifié à l'exécution, pas seulement commenté.

Une note de `v1` affirmait que purger « changerait l'ordre de sommation et ferait perdre la parité ». **Cette inférence est fausse** pour le cas qui nous occupe, et elle est retirée. Trois raisons, qui doivent toutes être vraies :

1. le corps de boucle est un *no-op* pour un ensemble vide — le `continue` précède la lecture de `book.due[borrower]`, donc aucune opération flottante n'a lieu ;

2. supprimer une clef d'un dict CPython *ne réordonne pas* les clefs restantes ; la séquence des débitrices non vides est identique ;
3. une entité morte est tuée et *ne peut plus jamais emprunter*, donc sa clef ne sera pas réinsérée.

Le piège à ne pas commettre. Purger les clefs vides des entités *vivantes* — celles dont tous les prêts se sont simplement clos — **casserait** la parité : un `defaultdict` réinsère la clef en *fin* d'ordre au prochain emprunt, ce qui change la séquence d'itération et donc l'ordre de sommation. C'est pourquoi la purge n'a lieu qu'à la mort, et pourquoi c'est écrit dans la docstring de la méthode.

4.3 La mesure

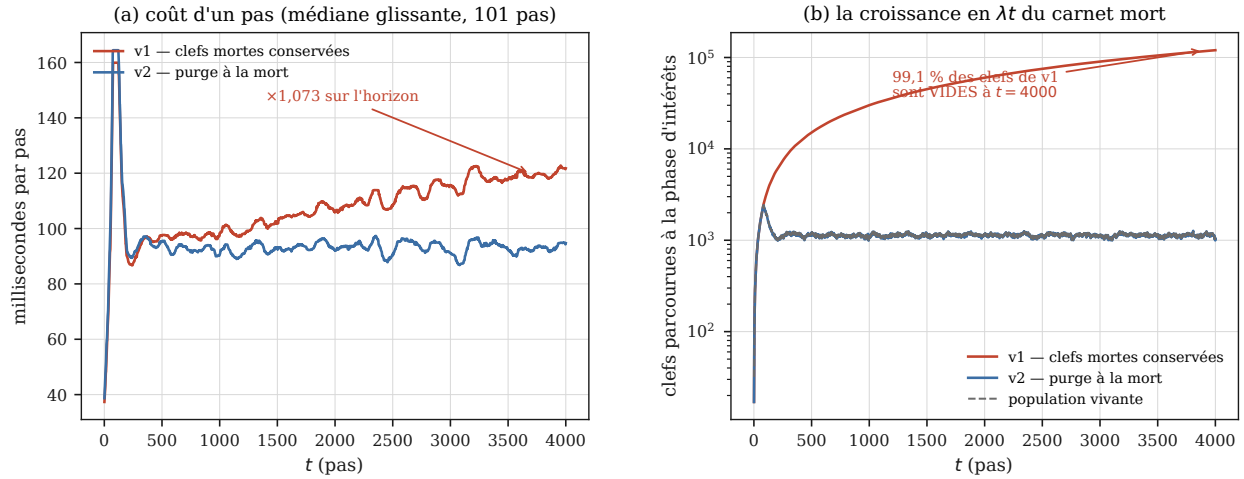


FIGURE 3 – **Le carnet mort de v1 croît en λt , et c'est ce qui rend le coût total quadratique en horizon.** 4000 pas, graine 0, régime de référence ; les deux moteurs tournent dans deux processus séparés, en parallèle, chacun sur son cœur. À gauche, le coût d'un pas : celui de v1 monte régulièrement, celui de v2 non. À droite, en échelle logarithmique, la grandeur qui l'explique — le nombre de clefs parcourues à la phase d'intérêts. Ce qu'il faut regarder est que la courbe de v2 se confond avec la population vivante (tireté), tandis que celle de v1 s'en détache dès les premiers pas et atteint deux ordres de grandeur au-dessus.

[Fait observé] Sur 4000 pas et à population constante, le carnet de v1 porte 120 153 clefs pour 1 031 entités vivantes : **99,1 % sont vides**. Celui de v2 en porte 1 023, soit la population vivante à une unité près. Le coût par pas de v1 est multiplié par 1,073 entre le début et la fin de l'horizon (111,6 à 119,8 ms) ; celui de v2 par 0,805, c'est-à-dire qu'il ne croît pas. Le run entier passe de 445 s à 383 s, soit **−13,9 %**.

[Incertitude] Le coût par pas de v2 *décroît* sur le même horizon. Ce n'est pas un effet de la purge : c'est le carnet *actif* qui se contracte après le transitoire initial, et v1 le subit aussi — chez lui, il est masqué par la croissance des clefs mortes. Le fait à retenir est que la taille du carnet mort ne pilote plus rien.

Ce que l'instrumentation coûte. La table ci-dessus décrit le moteur *livré*, instrumentation comprise. La même mesure faite sur le moteur du lot A seul — avant la tension native, les corrélations par entité et la sonde d'amplitude — donne 115,2 puis 92,3 ms/pas ($\times 0,801$) et 378 s au total, contre 115,6, 93,1 ms/pas et 383 s pour le moteur livré. **[Fait observé]** **L'instrumentation n'est pas mesurable** à ce niveau de bruit : l'écart est du même ordre que celui du moteur v1 entre les deux mesures (444 s puis 445 s), qui n'a pourtant pas changé d'une ligne.

5 La borne de transfert supprimée

Le paramètre `transfer_cap` n'avait que deux valeurs : `optimum`, l'énoncé littéral de l'institution, et `equalization`, un plafond du transfert à $(K_\ell - K_b)/2$. `equalization` est retiré de v2 : le tuple de validation, le champ de `Config`, la branche de plafonnement, le compteur `mkt_capped` et sa colonne, l'option de ligne de commande, et le champ exposé par l'interface.

[Fait observé] C'est du code mort, et c'est prouvé : `mkt_capped` cumulé vaut *exactement* 0 sur les runs v1 du dépôt. La suppression ne peut donc changer aucun nombre publié, et la parité doit être reconduite telle quelle. C'est précisément ce qui la distingue du chantier du sens (§3), qui change réellement le comportement, et c'est pourquoi elle est faite dans le premier lot, avant tout changement mesurable : la parité qui suit porte alors sur un moteur déjà simplifié.

Le pilote de plafond de v1 reste publié, et il faut noter la boucle. Le rapport de conception de v1 documente *pourquoi* le plafond est sans objet : les entités dopées deviennent en quelques dizaines de pas le côté riche de presque toutes leurs paires, l'optimum voudrait alors leur envoyer du capital, et l'ancienne règle de sens l'interdisait. [Inférence] Ce raisonnement s'appuie donc sur la règle que v2 supprime. Il reste valable comme justification historique, mais il ne peut plus servir d'argument sous le sens libre : sous le sens libre, ces paires *traitent*, en sens inverse. La question « le plafond redevient-il pertinent ? » est donc rouverte en droit. [Incertitude] Elle n'est pas tranchée ici, et le plafond n'est pas réintroduit : il faudrait pour cela un motif institutionnel, pas seulement la possibilité technique.

6 L'ordre des phases, en option

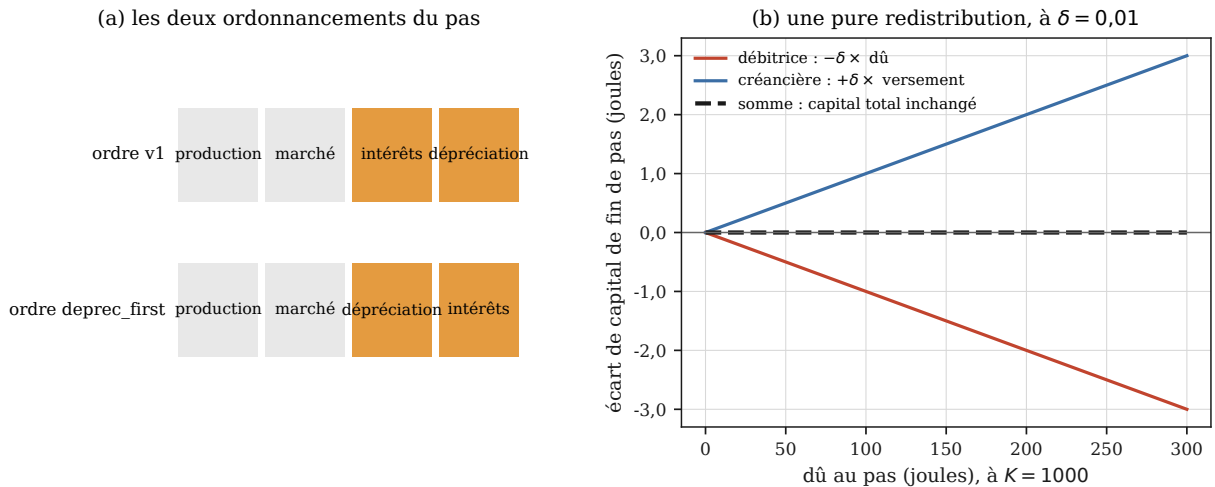


FIGURE 4 – **Échanger les deux dernières phases du pas conserve le capital total et le redistribue.** À gauche, les deux ordonnancements ; les deux blocs échangés sont en couleur. À droite, ce que l'échange fait au capital de fin de pas : une débitrice perd $\delta \times$ son dû, une créancière gagne $\delta \times$ son versement, et la somme — le trait noir — est plate à zéro. Ce qu'il faut regarder est que ce mécanisme n'est pas un choc mais un *flux* : il agit à chaque pas, sans jamais créer d'événement, ce qui explique qu'il déplace les agrégats sans changer le nombre de défauts.

Le champ `phase_order` vaut "v1" par défaut — production, service des intérêts, dépréciation — ou "deprec_first", qui inverse les deux dernières. La valeur par défaut préserve la parité sans détour ; l'autre ne la préserve pas, et c'est le but.

L'implémentation extrait les deux phases en fonctions (`_service_interest`, `_depreciate`) appelées dans l'ordre voulu. [Fait observé] L'extraction ne change aucun calcul : la parité bit à bit est reconduite après.

Une prédiction que le moteur calcule lui-même. Sous `deprec_first`, le capital disponible au moment de payer est réduit d'un facteur $1 - \delta$: bascule en défaut de liquidité toute débitrice dont le capital K vérifie $(1 - \delta)K < d_u \leq K$. C'est un ensemble que le moteur peut compter *dans le bras de référence lui-même* — c'est la colonne `defaults_window`. Aucune division n'est employée : $d_u \leq K/(1 - \delta)$ est écrit $d_u(1 - \delta) \leq K$. La prédiction est donc écrite *avant* que le bras traité n'existe, ce qui est le seul moment où une prédiction a une valeur.

7 L'instrumentation devenue native

7.1 La tension

v1 satisfaisait la demande de tension *a posteriori*, par un dérivé des séries, parce que son moteur était gelé. Il devait reconstituer l'effectif producteur par $n_{\text{vivantes}} + \text{morts}$, ce qui est exact tant qu'une seule technologie vit et approché sinon ; une colonne `basis` documentait laquelle des trois reconstructions avait servi.

v2 enregistre, par technologie et par pas, `n_prod` et `K_prod` — l'effectif et le capital à *l'instant de produire*, après le choc multiplicatif et avant que la production ne soit ajoutée au capital. La colonne `basis` disparaît, faute d'ambiguïté à documenter.

[Fait observé] Deux gains de justesse, et non seulement de commodité. Le rapport de Jensen $K_{\text{eq}}/\bar{K}$ devient exact, ses deux termes étant pris au même instant du pas, là où v1 mélangeait la production et le capital de fin de pas. Et le δ employé est celui du pas courant : un dérivé *a posteriori* lit un δ de fichier de configuration et ne verrait pas une intervention sur δ .

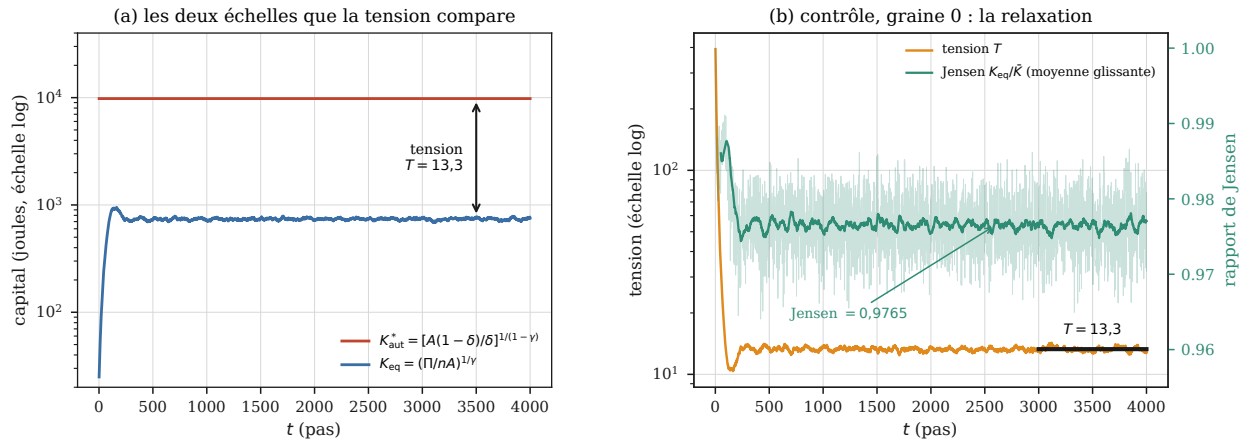


FIGURE 5 – **Ce que le calcul natif rend exact.** À gauche, les deux échelles que la tension compare, sur le bras de contrôle, graine 0. À droite, sa décroissance et le rapport de Jensen $K_{\text{eq}}/\bar{K}$ sur l'axe de droite. Ce qu'il faut regarder est ce dernier : il vaut 0,9765 en régime, soit un écart à 1 de plus de deux pour cent. C'est précisément la quantité que v1 devait estimer, son numérateur venant de la production et son dénominateur du capital de fin de pas — deux instants différents du même pas.

Le calcul vit dans `m4_3live_v2/tension.py`, module du *moteur* et non des scripts. Ce n'est pas un rangement : `scripts` est un nom de premier niveau générique, et les deux lignées en ont un ; un import par nom résoudreait vers l'une ou l'autre selon l'ordre de `sys.path`. Mettre le calcul dans le paquet supprime la collision par construction. Le module de tracé, lui, est chargé par chemin de fichier.

7.2 L'amplitude exacte

L'**amplitude** m d'une intervention est le facteur par lequel elle multiplie la production des entités qu'elle touche, au pas où elle agit pour la première fois :

$$m = \frac{\sum_{i \in \text{traitées}} A'_i K_i^{\gamma'_i}}{\sum_{i \in \text{traitées}} A_i K_i^{\gamma_i}}, \quad (9)$$

les deux sommes portant sur le *même* capital K_i , celui de l'instant de la production. v1 la reconstruisait depuis les agrégats; v2 pose une sonde $\{i \rightarrow (A_i, \gamma_i) \text{ d'avant}\}$ au moment de l'intervention et la consomme dans la phase de production du même pas.

Une correction que la mesure a imposée. [Fait observé] La sonde couvre aussi les entités *nées* au pas de l'intervention lorsque la technologie de naissance a changé. Sans elles, la part de production traitée d'une portée `all` vaut 0,9914 au lieu de 1 — mesuré avant correction. La raison est que ces entités portent déjà la nouvelle technologie mais n'auraient pas existé avec l'ancienne : les omettre du contrefactuel revient à leur attribuer la nouvelle production dans les deux branches.

Une identité, et pas une confirmation. Le rapport $E = (\Pi/\Pi_{\text{cf}} - 1)/((m - 1)p)$, où p est la part de production *ex ante* de la cohorte traitée, vaut 1 **par algèbre** : c'est une réécriture des définitions de m et de p . Son écart à 1 ne mesure que la propriété de l'aller-retour flottant. C'est écrit dans la docstring de `_close_amplitude` et redit dans le rapport de résultats, parce que présenter une identité comme une confirmation empirique est exactement l'erreur que v1 s'était signalée à elle-même.

7.3 La prédiction naïve du levier γ

Pour tout levier sur γ , l'amplitude mesurée est enregistrée à côté de deux prédictions : celle qu'on ferait en supposant $K = 1$ (où $K^{\Delta\gamma} = 1$, donc « le levier ne fait rien »), et celle qu'on ferait au capital équivalent, $K_{\text{eq}}^{\Delta\gamma}$.

[Fait observé] Sur un état de référence à $t = 200$, un levier $\gamma : 0,5 \rightarrow 0,6$ donne une amplitude mesurée de 1,960149, contre 1,000000 pour la prédiction naïve et 1,952475 pour la prédiction au capital équivalent (écart -0,39 %). C'est le contenu informatif de la mesure : un levier sur γ n'est pas un cadran de puissance, et l'échelle de capital du système suffit à le prédire à mieux que un demi pour cent.

[Fait observé] Ces valeurs sont engendrées par `scripts/amplitude.py`, qui relance les quatre leviers et écrit la mesure du moteur dans `results/analysis/amplitude.csv`. Elles étaient auparavant recopiées à la main depuis la sortie de `tests/test_amplitude.py` : un test *imprime* ses valeurs, il ne les persiste pas, de sorte que rien ne les rattachait à une source.

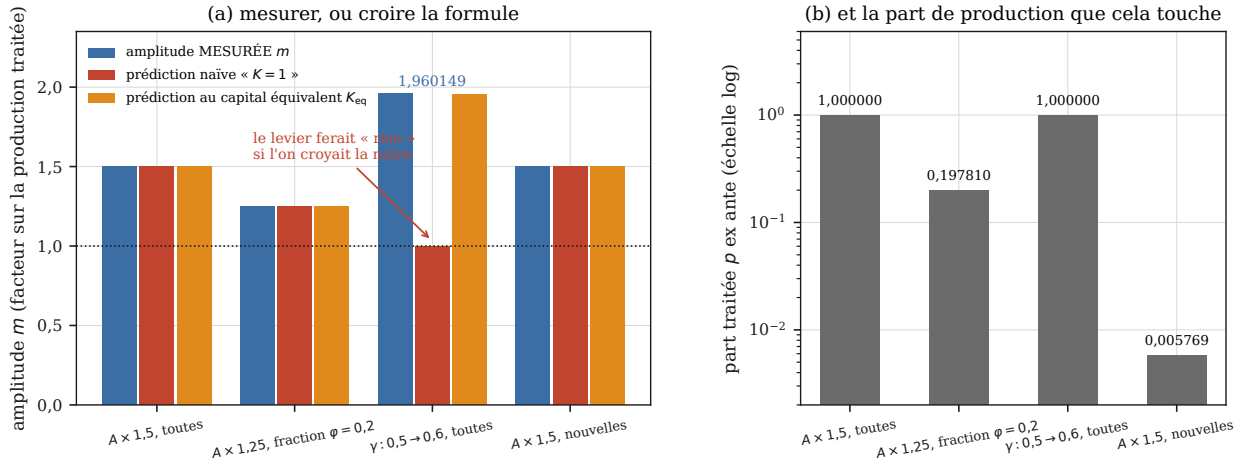


FIGURE 6 – Trois manières de répondre à « de combien ce levier agit-il ? », dont une seule est une mesure. À gauche, pour chacun des quatre leviers, l’amplitude enregistrée par le moteur, la prédiction naïve « $K = 1$ » et la prédiction au capital équivalent. Ce qu’il faut regarder est la troisième colonne : la prédiction naïve y annonce 1,000000, c’est-à-dire que le levier ne ferait rien. À droite, la part de production que chaque levier touche *ex ante*, en échelle logarithmique.

8 Les trois sémantiques de parité, et leur état

C’est le point le plus facile à saboter par inattention, et les trois sémantiques ne se confondent pas.

Parité obligatoire et bit-exacte (lot A). Supprimer *equalization* retire du code qui n’a jamais été exécuté ; purger les clefs mortes supprime un *no-op*. Toute déviation serait un bug de la suppression. [Fait observé] **Résultat : 8 000 pas × 26 colonnes contre m4_3_d1_baseline_seed0, écart maximal nul.** 9 366 225 appels au noyau, tous sur le chemin identité — exactement le nombre publié par v1. 739 s, contre 1077 s pour v1 sur le même test (*results/analysis/parity_lotA_full.log*).

Parité à retester, pas à supposer (lot B). En régime homogène l’optimum est le partage égal, donc $\delta = (K_b - K_a)/2 \geq 0$: le sens libre coïncide avec l’ancienne règle. Mais l’ordre du couple change l’ordre de sommation et d’insertion dans le carnet, et c’est le genre de détail qui décale un flottant. [Fait observé] **Résultat : parité reconduite, écart maximal nul**, sur le moteur v2 complet avec le sens libre par défaut (741 s, *results/analysis/parity_lotBCE_full.log*). **Et repassée une troisième fois sur le moteur effectivement livré**, après l’ajout de la sonde d’amplitude aux naissances et de la sonde de statistiques de marché : écart maximal nul, 9 369 224 appels au noyau — le même nombre, ce qui confirme qu’aucune de ces additions ne déplace un appel (747 s, *results/analysis/parity_final_full.log*).

Parité conditionnelle (lot E). Le défaut *phase_order* = "v1" doit la préserver — il la préserve, c’est la mesure ci-dessus. La branche "deprec_first" ne la préserve pas, par construction.

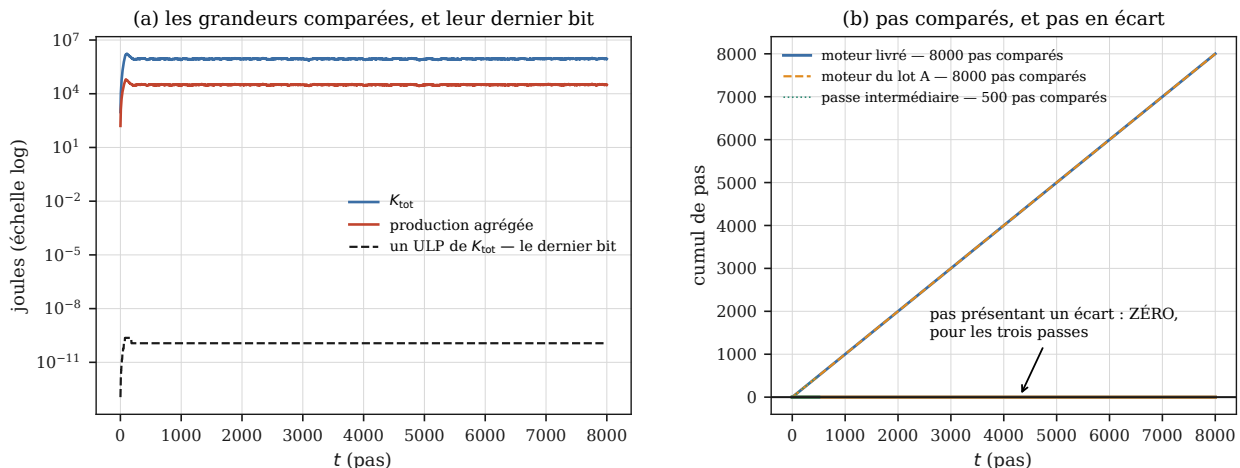


FIGURE 7 – L'écart à M4.3 n'est pas petit : il est nul, à chacun des 8 000 pas et sur chacune des 26 colonnes. À gauche, les grandeurs comparées et, en tireté, la valeur d'un seul bit de K_{tot} — l'échelle à laquelle une égalité flottante se joue. À droite, pour chacune des trois passes de parité, le cumul des pas comparés (qui monte) et le cumul des pas présentant un écart (qui reste plat à zéro, confondu avec l'axe). Une tolérance aurait produit une figure très différente : une bande étroite autour de zéro, et non zéro.

8.1 La seule différence, expliquée ligne à ligne

[Fait observé] Le moteur v2 fait 9 369 224 appels au noyau là où le lot A en faisait 9 366 225, soit +2999. Aucune tolérance n'a été employée pour absorber cet écart, parce qu'il ne porte sur aucun nombre.

L'explication est complète. v1 écartait les paires de capitaux égaux *avant* d'appeler le noyau (`if K[lender] <= K[borrower]: continue`). v2 sous le sens libre les lui soumet, parce qu'entre technologies différentes une paire de capitaux égaux a un optimum non trivial. En régime homogène, ces 2999 paires — des entités à capital nul, mises à zéro par la dépréciation — reçoivent $\delta^* = 0$, sont comptées `blocked_tiny` et ne traitent pas. Aucune opération flottante n'est ajoutée à la trajectoire, et aucune table du noyau n'est compilée différemment : le chemin identité ne tient pas de compteur d'usage.

8.2 La garantie qui compte vraiment pour la campagne

[Inférence] La parité avec M4.3 ne dit rien du régime hétérogène, qui est justement celui où le sens du prêt a un objet. La garantie nécessaire est autre : sous `loan_direction="richest_lends"`, le moteur v2 doit reproduire le moteur v1 *y compris* quand deux technologies coexistent. Sans quoi la campagne A/B ne compare pas deux règles, mais deux lignées de code.

[Fait observé] C'est vérifié : 300 pas $\times$ **34 colonnes**, intervention $A \times 1,5$ sur 20 % de la population à $t = 100$, deux technologies vivantes, 61 166 paires refusées pour cause de sens — **écart maximal NUL** entre le moteur v1 et v2 sous `richest_lends`. Contrôle négatif dans le même test : sous le sens libre, la première divergence tombe au pas de l'intervention lui-même, et 30 768 prêts sont conclus dans le sens interdit.

9 Ce que la suite de tests couvre

Assertions Python simples, convention du dépôt — pas de `pytest`.

fichier	ce qu'il établit
<code>test_parity_m4_3.py</code>	parité bit à bit avec M4.3, 26 colonnes ; 500 pas par défaut, 8000 avec <code>-full</code>
<code>test_v1_equivalence.py</code>	v2 sous <code>richest_lends</code> reproduit le moteur v1 en régime hétérogène ; contrôle négatif sous le sens libre
<code>test_loan_direction.py</code>	symétrie bit à bit du taux ; une paire construite à la main où l'optimum exige que la moins riche cède ; le carnet, la valeur nette et le compteur contrefactuel
<code>test_tension.py</code>	formes fermées de K_{aut}^* et K_{eq} ; Jensen = 1 exactement à capitaux égaux ; <code>n_prod</code> = vivantes + morts sur 400 pas ; agrégat pondéré par la production
<code>test_amplitude.py</code>	amplitude exacte sur quatre leviers, part traitée, prédiction naïve, et recalcul indépendant depuis les productions enregistrées
<code>test_scale_covariance.py</code>	covariance d'échelle : population et prêts identiques, extensives à 3×10^{-14} près, à deux γ
<code>test_institution.py</code>	les trois régimes du noyau contre forme fermée et Newton ; ordre 4 de la table
<code>test_scopes.py</code>	sémantique des portées, y compris les combinaisons refusées
<code>test_replay.py</code>	rejeu d'une session en direct réelle, invariant de pause, pas-à-pas, aller-retour de snapshot
<code>test_resume_divergence.py</code>	reprise d'un run M4.3 stocké, écart mesuré et non supposé
<code>test_surplus_rate.py</code>	règle de taux alternative $r = p\Delta/q$
<code>test_web_live.py</code>	l'interface <code>/live2</code> de bout en bout par HTTP, et la non-régression des routes de <code>simulation_lab</code> et de v1

[Incertitude] `test_web_live.py` n'ouvre pas de navigateur et ne regarde pas les pixels. Il démarre le serveur, crée une session, la fait tourner, lui soumet une intervention, relit l'état par la route que le navigateur emploie, et vérifie que chaque clef et chaque colonne que le JavaScript consomme est présente. Le rendu graphique lui-même n'est pas testé.

Un défaut trouvé par ce test, et pas par la lecture. **[Fait observé]** Le préfixe des routes POST du routeur v2 était resté [`"api"`, `"live"`, `"sessions"`] après le renommage vers `/live2`, ce qui rendait *toutes* les actions de session — jouer, mettre en pause, intervenir, écrire — inaccessibles. Les routes GET fonctionnaient, si bien qu'une vérification par lecture de page n'aurait rien vu.

10 Les décisions laissées ouvertes, et comment elles ont été tranchées

question	décision	ce qui la justifie
Nom du paquet forké	<code>m4_3live_v2</code>	Aucun conflit ; permet d'importer les deux moteurs dans un même processus.
Le taux quand la créancière est la plus fragile	Règle inchangée	La formule est symétrique bit à bit : elle ne contient aucun rôle (§3.3). L'alternative « rendement marginal commun d'après l'échange » est documentée et écartée.
Nombre de graines de verdict	12	Coût <i>mesuré</i> et non extrapolé : une cellule pilote de 2000 pas coûte 144,5 s sous le sens libre et 140,8 s sous la règle v1. La campagne entière tient en une heure sur 7 processus. Seuil de Student à 11 degrés de liberté : 2,201 à 5 %, contre 2,776 à 4 degrés.
Rééchelonner <code>MIN_LOAN</code> et <code>ZERO_TOL</code>	Non	Cela rendrait la covariance d'échelle exacte, mais casserait la comparabilité directe avec toutes les campagnes antérieures. Le gain ne se voit qu'au quatorzième chiffre : l'écart résiduel mesuré est $3,1 \times 10^{-14}$ (<code>tests/test_scale_covariance.py</code>).
La covariance de pas de temps devient-elle un test de non-régression ?	Non — mais la covariance d'échelle, oui	Le résidu du recalage temporel est de 6 à 17 %, et sa source est la phase de marché, qui ne se compose pas : un seuil y serait arbitraire. La covariance d'échelle, elle, est exacte à 3×10^{-14} : elle fait un test légitime, et c'est <code>test_scale_covariance.py</code> .

11 Un trou assumé : les annotations de conception de v1

La feuille de route de v2 réserve une section aux annotations que l'utilisateur a portées sur le rapport de conception de v1. **[Fait observé]** Ce fichier ne porte *aucune* annotation sur le disque : zéro objet `/Subtype/Text`, vérifié par `pdftk dump_data_annots` et par `mutool show`. L'horodatage du fichier est celui de sa compilation, antérieur à la première note du rapport final.

[Inférence] Les annotations n'ont vraisemblablement pas été enregistrées depuis la visionneuse. Rien n'a été inventé pour combler ce trou. Si une copie annotée est fournie, elle sera intégrée ; en attendant, le présent document est **incomplet d'un jeu de retours**, et c'est dit ici plutôt que passé sous silence.

12 Périmètre : ce qui a été retiré, et à la demande de qui

- **Aucun traitement du cas partiel.** La portée `fraction` reste dans le moteur — elle est testée, elle fonctionne — mais aucune campagne de v2 ne l'emploie. Sortent avec elle : les bras `null` appariés, l'inversion de la part traitée *ex post* en part *ex ante*, et la question du long horizon posée en portée partielle. v2 mesure en portée globale, où l'intensité de traitement vaut 1 par construction.
- **Une seule borne de transfert.** §5.

Les résultats de v1 obtenus en portée partielle restent acquis et publiés ; ils ne sont simplement pas prolongés.

13 Un écart au séquençement demandé, et sa portée

[Incertitude] Le prompt demande que les lots qui changent le modèle soient *séparés dans l'historique git*. L'historique de v2 comporte le fork, puis le lot A seul, puis les lots B, C et E groupés dans un même commit — ils ont été écrits dans une même passe d'édition sur un seul fichier.

Ce qui est en jeu est l'attribution des verdicts, et elle n'en dépend pas : elle est assurée par des **drapeaux d'exécution**. La campagne du sens du prêt oppose `loan_direction="free"` à `"richest_lends"` dans le *même binaire*, sur les *mêmes* snapshots et les *mêmes* graines ; celle de l'ordre des phases oppose de même `phase_order="v1"` à `"deprec_first"`. C'est plus fort qu'une séparation d'historique, qui ne garantirait qu'une chose : que les deux versions ont existé séparément. Le lot C, lui, est de l'instrumentation et ne change aucune trajectoire — la parité bit à bit le vérifie.

14 Reproduire

Toutes les commandes partent de `/home/anatole/jupyter` avec `PY=/home/anatole/jupyter/.venv/bin/py`

commande	coût mesuré
<code>\$PY m4_3live_v2_credit_soc/tests/test_parity_m4_3.py -full</code>	741 s
<code>\$PY m4_3live_v2_credit_soc/tests/test_v1_equivalence.py</code>	59 s
<code>\$PY m4_3live_v2_credit_soc/tests/test_loan_direction.py</code>	< 1 s
<code>\$PY m4_3live_v2_credit_soc/tests/test_tension.py</code>	25 s
<code>\$PY m4_3live_v2_credit_soc/tests/test_amplitude.py</code>	60 s
<code>\$PY m4_3live_v2_credit_soc/tests/test_scale_covariance.py</code>	200 s
<code>\$PY m4_3live_v2_credit_soc/tests/test_web_live.py</code>	30 s
<code>\$PY m4_3live_v2_credit_soc/scripts/cost_profile.py -steps 4000</code>	447 s (2 processus)
<code>\$PY m4_3live_v2_credit_soc/scripts/amplitude.py</code>	102 s
<code>\$PY m4_3live_v2_credit_soc/scripts/make_figures.py</code>	26 s (23 figures)
<code>\$PY m4_3live_v2_credit_soc/tests/test_figures.py</code>	< 1 s

Le protocole des campagnes et le coût de chacune sont donnés dans le rapport de résultats, section « Reproduire ».

A Annexe de traçabilité

Toute simulation citée dans ce document est identifiable par son `run_id` dans `simulation_lab`. Les renvois sont des *sections* et jamais des numéros de figure. Le tableau est engendré par `scripts/make_traceability.py`, qui refuse de produire une annexe dont une ligne serait sans rôle.

run_id (simulation_lab)	groupe	apparaît dans	rôle / interventions
<code>m4_3live_v2_arm_free_all_A150__seed0</code>	lot D	campagne A/B	hausse globale $A \times 1.5$: reste homogène, donc insensible à la règle de sens
<i>t=2001 A→1.5 (all)</i> <code>m4_3live_v2_arm_free_all_A150__seed1</code>	lot D	campagne A/B	hausse globale $A \times 1.5$: reste homogène, donc insensible à la règle de sens
<i>t=2001 A→1.5 (all)</i> <code>m4_3live_v2_arm_free_all_A150__seed10</code>	lot D	campagne A/B	hausse globale $A \times 1.5$: reste homogène, donc insensible à la règle de sens
<i>t=2001 A→1.5 (all)</i>			

run_id (simulation_lab)	groupe	apparaît dans	rôle / interventions
m4_3live_v2__arm__free__all_A150__seed11 <i>t=2001 A→1.5 (all)</i>	lot D	campagne A/B	hausse globale $A \backslash times 1,5$: reste homogène, donc insensible à la règle de sens
m4_3live_v2__arm__free__all_A150__seed2 <i>t=2001 A→1.5 (all)</i>	lot D	campagne A/B	hausse globale $A \backslash times 1,5$: reste homogène, donc insensible à la règle de sens
m4_3live_v2__arm__free__all_A150__seed3 <i>t=2001 A→1.5 (all)</i>	lot D	campagne A/B	hausse globale $A \backslash times 1,5$: reste homogène, donc insensible à la règle de sens
m4_3live_v2__arm__free__all_A150__seed4 <i>t=2001 A→1.5 (all)</i>	lot D	campagne A/B	hausse globale $A \backslash times 1,5$: reste homogène, donc insensible à la règle de sens
m4_3live_v2__arm__free__all_A150__seed5 <i>t=2001 A→1.5 (all)</i>	lot D	campagne A/B	hausse globale $A \backslash times 1,5$: reste homogène, donc insensible à la règle de sens
m4_3live_v2__arm__free__all_A150__seed6 <i>t=2001 A→1.5 (all)</i>	lot D	campagne A/B	hausse globale $A \backslash times 1,5$: reste homogène, donc insensible à la règle de sens
m4_3live_v2__arm__free__all_A150__seed7 <i>t=2001 A→1.5 (all)</i>	lot D	campagne A/B	hausse globale $A \backslash times 1,5$: reste homogène, donc insensible à la règle de sens
m4_3live_v2__arm__free__all_A150__seed8 <i>t=2001 A→1.5 (all)</i>	lot D	campagne A/B	hausse globale $A \backslash times 1,5$: reste homogène, donc insensible à la règle de sens
m4_3live_v2__arm__free__all_A150__seed9 <i>t=2001 A→1.5 (all)</i>	lot D	campagne A/B	hausse globale $A \backslash times 1,5$: reste homogène, donc insensible à la règle de sens
m4_3live_v2__arm__free__control__seed0	lot D	campagne A/B ; contrôle de stationnarité	référence inerte sous sens libre
m4_3live_v2__arm__free__control__seed1	lot D	campagne A/B ; contrôle de stationnarité	référence inerte sous sens libre
m4_3live_v2__arm__free__control__seed10	lot D	campagne A/B ; contrôle de stationnarité	référence inerte sous sens libre
m4_3live_v2__arm__free__control__seed11	lot D	campagne A/B ; contrôle de stationnarité	référence inerte sous sens libre
m4_3live_v2__arm__free__control__seed2	lot D	campagne A/B ; contrôle de stationnarité	référence inerte sous sens libre
m4_3live_v2__arm__free__control__seed3	lot D	campagne A/B ; contrôle de stationnarité	référence inerte sous sens libre
m4_3live_v2__arm__free__control__seed4	lot D	campagne A/B ; contrôle de stationnarité	référence inerte sous sens libre
m4_3live_v2__arm__free__control__seed5	lot D	campagne A/B ; contrôle de stationnarité	référence inerte sous sens libre
m4_3live_v2__arm__free__control__seed6	lot D	campagne A/B ; contrôle de stationnarité	référence inerte sous sens libre
m4_3live_v2__arm__free__control__seed7	lot D	campagne A/B ; contrôle de stationnarité	référence inerte sous sens libre
m4_3live_v2__arm__free__control__seed8	lot D	campagne A/B ; contrôle de stationnarité	référence inerte sous sens libre
m4_3live_v2__arm__free__control__seed9	lot D	campagne A/B ; contrôle de stationnarité	référence inerte sous sens libre
m4_3live_v2__arm__free__new_A075__seed0 <i>t=2001 A→0.75 (new)</i>	lot D	campagne A/B	vintage $A \backslash times 0,75$, sens libre
m4_3live_v2__arm__free__new_A075__seed1 <i>t=2001 A→0.75 (new)</i>	lot D	campagne A/B	vintage $A \backslash times 0,75$, sens libre
m4_3live_v2__arm__free__new_A075__seed10 <i>t=2001 A→0.75 (new)</i>	lot D	campagne A/B	vintage $A \backslash times 0,75$, sens libre
m4_3live_v2__arm__free__new_A075__seed11 <i>t=2001 A→0.75 (new)</i>	lot D	campagne A/B	vintage $A \backslash times 0,75$, sens libre

run_id (simulation_lab)	groupe	apparaît dans	rôle / interventions
m4_3live_v2__arm__free__new_A075__seed2 <i>t=2001 A→0.75 (new)</i>	lot D	campagne A/B	vintage $A \backslash times 0,75$, sens libre
m4_3live_v2__arm__free__new_A075__seed3 <i>t=2001 A→0.75 (new)</i>	lot D	campagne A/B	vintage $A \backslash times 0,75$, sens libre
m4_3live_v2__arm__free__new_A075__seed4 <i>t=2001 A→0.75 (new)</i>	lot D	campagne A/B	vintage $A \backslash times 0,75$, sens libre
m4_3live_v2__arm__free__new_A075__seed5 <i>t=2001 A→0.75 (new)</i>	lot D	campagne A/B	vintage $A \backslash times 0,75$, sens libre
m4_3live_v2__arm__free__new_A075__seed6 <i>t=2001 A→0.75 (new)</i>	lot D	campagne A/B	vintage $A \backslash times 0,75$, sens libre
m4_3live_v2__arm__free__new_A075__seed7 <i>t=2001 A→0.75 (new)</i>	lot D	campagne A/B	vintage $A \backslash times 0,75$, sens libre
m4_3live_v2__arm__free__new_A075__seed8 <i>t=2001 A→0.75 (new)</i>	lot D	campagne A/B	vintage $A \backslash times 0,75$, sens libre
m4_3live_v2__arm__free__new_A075__seed9 <i>t=2001 A→0.75 (new)</i>	lot D	campagne A/B	vintage $A \backslash times 0,75$, sens libre
m4_3live_v2__arm__free__new_A150__seed0 <i>t=2001 A→1.5 (new)</i>	lot D	campagne A/B ; régime nouveau	vintage $A \backslash times 1,5$, sens libre
m4_3live_v2__arm__free__new_A150__seed1 <i>t=2001 A→1.5 (new)</i>	lot D	campagne A/B ; régime nouveau	vintage $A \backslash times 1,5$, sens libre
m4_3live_v2__arm__free__new_A150__seed10 <i>t=2001 A→1.5 (new)</i>	lot D	campagne A/B ; régime nouveau	vintage $A \backslash times 1,5$, sens libre
m4_3live_v2__arm__free__new_A150__seed11 <i>t=2001 A→1.5 (new)</i>	lot D	campagne A/B ; régime nouveau	vintage $A \backslash times 1,5$, sens libre
m4_3live_v2__arm__free__new_A150__seed2 <i>t=2001 A→1.5 (new)</i>	lot D	campagne A/B ; régime nouveau	vintage $A \backslash times 1,5$, sens libre
m4_3live_v2__arm__free__new_A150__seed3 <i>t=2001 A→1.5 (new)</i>	lot D	campagne A/B ; régime nouveau	vintage $A \backslash times 1,5$, sens libre
m4_3live_v2__arm__free__new_A150__seed4 <i>t=2001 A→1.5 (new)</i>	lot D	campagne A/B ; régime nouveau	vintage $A \backslash times 1,5$, sens libre
m4_3live_v2__arm__free__new_A150__seed5 <i>t=2001 A→1.5 (new)</i>	lot D	campagne A/B ; régime nouveau	vintage $A \backslash times 1,5$, sens libre
m4_3live_v2__arm__free__new_A150__seed6 <i>t=2001 A→1.5 (new)</i>	lot D	campagne A/B ; régime nouveau	vintage $A \backslash times 1,5$, sens libre
m4_3live_v2__arm__free__new_A150__seed7 <i>t=2001 A→1.5 (new)</i>	lot D	campagne A/B ; régime nouveau	vintage $A \backslash times 1,5$, sens libre
m4_3live_v2__arm__free__new_A150__seed8 <i>t=2001 A→1.5 (new)</i>	lot D	campagne A/B ; régime nouveau	vintage $A \backslash times 1,5$, sens libre
m4_3live_v2__arm__free__new_A150__seed9 <i>t=2001 A→1.5 (new)</i>	lot D	campagne A/B ; régime nouveau	vintage $A \backslash times 1,5$, sens libre
m4_3live_v2__arm__free__new_g060__seed0 <i>t=2001 gamma→0.6 (new)</i>	lot D	campagne A/B	vintage $\backslash gamma : 0,5 \backslash to 0,6$, sens libre ; exerce le régime (c) du noyau
m4_3live_v2__arm__free__new_g060__seed1 <i>t=2001 gamma→0.6 (new)</i>	lot D	campagne A/B	vintage $\backslash gamma : 0,5 \backslash to 0,6$, sens libre ; exerce le régime (c) du noyau
m4_3live_v2__arm__free__new_g060__seed10 <i>t=2001 gamma→0.6 (new)</i>	lot D	campagne A/B	vintage $\backslash gamma : 0,5 \backslash to 0,6$, sens libre ; exerce le régime (c) du noyau
m4_3live_v2__arm__free__new_g060__seed11 <i>t=2001 gamma→0.6 (new)</i>	lot D	campagne A/B	vintage $\backslash gamma : 0,5 \backslash to 0,6$, sens libre ; exerce le régime (c) du noyau

run_id (simulation_lab)	groupe	apparaît dans	rôle / interventions
m4_3live_v2__arm__free__new_g060__seed2 <i>t=2001 gamma→0.6 (new)</i>	lot D	campagne A/B	vintage $\backslash gamma : 0,5 \backslash to 0,6$, sens libre; exerce le régime (c) du noyau
m4_3live_v2__arm__free__new_g060__seed3 <i>t=2001 gamma→0.6 (new)</i>	lot D	campagne A/B	vintage $\backslash gamma : 0,5 \backslash to 0,6$, sens libre; exerce le régime (c) du noyau
m4_3live_v2__arm__free__new_g060__seed4 <i>t=2001 gamma→0.6 (new)</i>	lot D	campagne A/B	vintage $\backslash gamma : 0,5 \backslash to 0,6$, sens libre; exerce le régime (c) du noyau
m4_3live_v2__arm__free__new_g060__seed5 <i>t=2001 gamma→0.6 (new)</i>	lot D	campagne A/B	vintage $\backslash gamma : 0,5 \backslash to 0,6$, sens libre; exerce le régime (c) du noyau
m4_3live_v2__arm__free__new_g060__seed6 <i>t=2001 gamma→0.6 (new)</i>	lot D	campagne A/B	vintage $\backslash gamma : 0,5 \backslash to 0,6$, sens libre; exerce le régime (c) du noyau
m4_3live_v2__arm__free__new_g060__seed7 <i>t=2001 gamma→0.6 (new)</i>	lot D	campagne A/B	vintage $\backslash gamma : 0,5 \backslash to 0,6$, sens libre; exerce le régime (c) du noyau
m4_3live_v2__arm__free__new_g060__seed8 <i>t=2001 gamma→0.6 (new)</i>	lot D	campagne A/B	vintage $\backslash gamma : 0,5 \backslash to 0,6$, sens libre; exerce le régime (c) du noyau
m4_3live_v2__arm__free__new_g060__seed9 <i>t=2001 gamma→0.6 (new)</i>	lot D	campagne A/B	vintage $\backslash gamma : 0,5 \backslash to 0,6$, sens libre; exerce le régime (c) du noyau
m4_3live_v2__arm__richest_lends__new_A075__seed0 <i>t=2001 A→0.75 (new)</i>	lot D	campagne A/B	même bras sous la règle v1
m4_3live_v2__arm__richest_lends__new_A075__seed1 <i>t=2001 A→0.75 (new)</i>	lot D	campagne A/B	même bras sous la règle v1
m4_3live_v2__arm__richest_lends__new_A075__seed10 <i>t=2001 A→0.75 (new)</i>	lot D	campagne A/B	même bras sous la règle v1
m4_3live_v2__arm__richest_lends__new_A075__seed11 <i>t=2001 A→0.75 (new)</i>	lot D	campagne A/B	même bras sous la règle v1
m4_3live_v2__arm__richest_lends__new_A075__seed2 <i>t=2001 A→0.75 (new)</i>	lot D	campagne A/B	même bras sous la règle v1
m4_3live_v2__arm__richest_lends__new_A075__seed3 <i>t=2001 A→0.75 (new)</i>	lot D	campagne A/B	même bras sous la règle v1
m4_3live_v2__arm__richest_lends__new_A075__seed4 <i>t=2001 A→0.75 (new)</i>	lot D	campagne A/B	même bras sous la règle v1
m4_3live_v2__arm__richest_lends__new_A075__seed5 <i>t=2001 A→0.75 (new)</i>	lot D	campagne A/B	même bras sous la règle v1
m4_3live_v2__arm__richest_lends__new_A075__seed6 <i>t=2001 A→0.75 (new)</i>	lot D	campagne A/B	même bras sous la règle v1
m4_3live_v2__arm__richest_lends__new_A075__seed7 <i>t=2001 A→0.75 (new)</i>	lot D	campagne A/B	même bras sous la règle v1
m4_3live_v2__arm__richest_lends__new_A075__seed8 <i>t=2001 A→0.75 (new)</i>	lot D	campagne A/B	même bras sous la règle v1
m4_3live_v2__arm__richest_lends__new_A075__seed9 <i>t=2001 A→0.75 (new)</i>	lot D	campagne A/B	même bras sous la règle v1
m4_3live_v2__arm__richest_lends__new_A150__seed0 <i>t=2001 A→1.5 (new)</i>	lot D	campagne A/B	même bras sous la règle v1 « la plus riche prête »
m4_3live_v2__arm__richest_lends__new_A150__seed1 <i>t=2001 A→1.5 (new)</i>	lot D	campagne A/B	même bras sous la règle v1 « la plus riche prête »
m4_3live_v2__arm__richest_lends__new_A150__seed10 <i>t=2001 A→1.5 (new)</i>	lot D	campagne A/B	même bras sous la règle v1 « la plus riche prête »
m4_3live_v2__arm__richest_lends__new_A150__seed11 <i>t=2001 A→1.5 (new)</i>	lot D	campagne A/B	même bras sous la règle v1 « la plus riche prête »

run_id (simulation_lab)	groupe	apparaît dans	rôle / interventions
m4_3live_v2__arm__richest_lends__new_A150__seed2 <i>t=2001 A→1.5 (new)</i>	lot D	campagne A/B	même bras sous la règle v1 « la plus riche prête »
m4_3live_v2__arm__richest_lends__new_A150__seed3 <i>t=2001 A→1.5 (new)</i>	lot D	campagne A/B	même bras sous la règle v1 « la plus riche prête »
m4_3live_v2__arm__richest_lends__new_A150__seed4 <i>t=2001 A→1.5 (new)</i>	lot D	campagne A/B	même bras sous la règle v1 « la plus riche prête »
m4_3live_v2__arm__richest_lends__new_A150__seed5 <i>t=2001 A→1.5 (new)</i>	lot D	campagne A/B	même bras sous la règle v1 « la plus riche prête »
m4_3live_v2__arm__richest_lends__new_A150__seed6 <i>t=2001 A→1.5 (new)</i>	lot D	campagne A/B	même bras sous la règle v1 « la plus riche prête »
m4_3live_v2__arm__richest_lends__new_A150__seed7 <i>t=2001 A→1.5 (new)</i>	lot D	campagne A/B	même bras sous la règle v1 « la plus riche prête »
m4_3live_v2__arm__richest_lends__new_A150__seed8 <i>t=2001 A→1.5 (new)</i>	lot D	campagne A/B	même bras sous la règle v1 « la plus riche prête »
m4_3live_v2__arm__richest_lends__new_A150__seed9 <i>t=2001 A→1.5 (new)</i>	lot D	campagne A/B	même bras sous la règle v1 « la plus riche prête »
m4_3live_v2__arm__richest_lends__new_g060__seed0 <i>t=2001 gamma→0.6 (new)</i>	lot D	campagne A/B	même bras sous la règle v1
m4_3live_v2__arm__richest_lends__new_g060__seed1 <i>t=2001 gamma→0.6 (new)</i>	lot D	campagne A/B	même bras sous la règle v1
m4_3live_v2__arm__richest_lends__new_g060__seed10 <i>t=2001 gamma→0.6 (new)</i>	lot D	campagne A/B	même bras sous la règle v1
m4_3live_v2__arm__richest_lends__new_g060__seed11 <i>t=2001 gamma→0.6 (new)</i>	lot D	campagne A/B	même bras sous la règle v1
m4_3live_v2__arm__richest_lends__new_g060__seed2 <i>t=2001 gamma→0.6 (new)</i>	lot D	campagne A/B	même bras sous la règle v1
m4_3live_v2__arm__richest_lends__new_g060__seed3 <i>t=2001 gamma→0.6 (new)</i>	lot D	campagne A/B	même bras sous la règle v1
m4_3live_v2__arm__richest_lends__new_g060__seed4 <i>t=2001 gamma→0.6 (new)</i>	lot D	campagne A/B	même bras sous la règle v1
m4_3live_v2__arm__richest_lends__new_g060__seed5 <i>t=2001 gamma→0.6 (new)</i>	lot D	campagne A/B	même bras sous la règle v1
m4_3live_v2__arm__richest_lends__new_g060__seed6 <i>t=2001 gamma→0.6 (new)</i>	lot D	campagne A/B	même bras sous la règle v1
m4_3live_v2__arm__richest_lends__new_g060__seed7 <i>t=2001 gamma→0.6 (new)</i>	lot D	campagne A/B	même bras sous la règle v1
m4_3live_v2__arm__richest_lends__new_g060__seed8 <i>t=2001 gamma→0.6 (new)</i>	lot D	campagne A/B	même bras sous la règle v1
m4_3live_v2__arm__richest_lends__new_g060__seed9 <i>t=2001 gamma→0.6 (new)</i>	lot D	campagne A/B	même bras sous la règle v1
m4_3live_v2__burn__seed0	amorçage	protocole; source des snapshots à $t_0 = 2000$ de tous les bras	amorçage partagé $0 \rightarrow 2000$, homogène, commun aux deux règles de sens
m4_3live_v2__burn__seed1	amorçage	protocole; source des snapshots à $t_0 = 2000$ de tous les bras	amorçage partagé $0 \rightarrow 2000$, homogène, commun aux deux règles de sens
m4_3live_v2__burn__seed10	amorçage	protocole; source des snapshots à $t_0 = 2000$ de tous les bras	amorçage partagé $0 \rightarrow 2000$, homogène, commun aux deux règles de sens
m4_3live_v2__burn__seed11	amorçage	protocole; source des snapshots à $t_0 = 2000$ de tous les bras	amorçage partagé $0 \rightarrow 2000$, homogène, commun aux deux règles de sens

run_id (simulation_lab)	groupe	apparaît dans	rôle / interventions
m4_3live_v2__burn__seed2	amorçage	protocole ; source des snapshots à $t_0 = 2000$ de tous les bras	amorçage partagé $0 \rightarrow 2000$, homogène, commun aux deux règles de sens
m4_3live_v2__burn__seed3	amorçage	protocole ; source des snapshots à $t_0 = 2000$ de tous les bras	amorçage partagé $0 \rightarrow 2000$, homogène, commun aux deux règles de sens
m4_3live_v2__burn__seed4	amorçage	protocole ; source des snapshots à $t_0 = 2000$ de tous les bras	amorçage partagé $0 \rightarrow 2000$, homogène, commun aux deux règles de sens
m4_3live_v2__burn__seed5	amorçage	protocole ; source des snapshots à $t_0 = 2000$ de tous les bras	amorçage partagé $0 \rightarrow 2000$, homogène, commun aux deux règles de sens
m4_3live_v2__burn__seed6	amorçage	protocole ; source des snapshots à $t_0 = 2000$ de tous les bras	amorçage partagé $0 \rightarrow 2000$, homogène, commun aux deux règles de sens
m4_3live_v2__burn__seed7	amorçage	protocole ; source des snapshots à $t_0 = 2000$ de tous les bras	amorçage partagé $0 \rightarrow 2000$, homogène, commun aux deux règles de sens
m4_3live_v2__burn__seed8	amorçage	protocole ; source des snapshots à $t_0 = 2000$ de tous les bras	amorçage partagé $0 \rightarrow 2000$, homogène, commun aux deux règles de sens
m4_3live_v2__burn__seed9	amorçage	protocole ; source des snapshots à $t_0 = 2000$ de tous les bras	amorçage partagé $0 \rightarrow 2000$, homogène, commun aux deux règles de sens
m4_3live_v2__phase__deprec_first__seed0	lot E	ordre des phases	dépréciation avant service des intérêts, apparié au contrôle
m4_3live_v2__phase__deprec_first__seed1	lot E	ordre des phases	dépréciation avant service des intérêts, apparié au contrôle
m4_3live_v2__phase__deprec_first__seed10	lot E	ordre des phases	dépréciation avant service des intérêts, apparié au contrôle
m4_3live_v2__phase__deprec_first__seed11	lot E	ordre des phases	dépréciation avant service des intérêts, apparié au contrôle
m4_3live_v2__phase__deprec_first__seed2	lot E	ordre des phases	dépréciation avant service des intérêts, apparié au contrôle
m4_3live_v2__phase__deprec_first__seed3	lot E	ordre des phases	dépréciation avant service des intérêts, apparié au contrôle
m4_3live_v2__phase__deprec_first__seed4	lot E	ordre des phases	dépréciation avant service des intérêts, apparié au contrôle
m4_3live_v2__phase__deprec_first__seed5	lot E	ordre des phases	dépréciation avant service des intérêts, apparié au contrôle
m4_3live_v2__phase__deprec_first__seed6	lot E	ordre des phases	dépréciation avant service des intérêts, apparié au contrôle
m4_3live_v2__phase__deprec_first__seed7	lot E	ordre des phases	dépréciation avant service des intérêts, apparié au contrôle
m4_3live_v2__phase__deprec_first__seed8	lot E	ordre des phases	dépréciation avant service des intérêts, apparié au contrôle
m4_3live_v2__phase__deprec_first__seed9	lot E	ordre des phases	dépréciation avant service des intérêts, apparié au contrôle
m4_3live_v2__rotation__K0_100__seed0	lot F	rotation du crédit (décomposition et fermeture)	levier K_0 (capital de naissance) seul (= 100)
m4_3live_v2__rotation__K0_100__seed1	lot F	rotation du crédit (décomposition et fermeture)	levier K_0 (capital de naissance) seul (= 100)
m4_3live_v2__rotation__K0_100__seed2	lot F	rotation du crédit (décomposition et fermeture)	levier K_0 (capital de naissance) seul (= 100)
m4_3live_v2__rotation__K0_6.25__seed0	lot F	rotation du crédit (décomposition et fermeture)	levier K_0 (capital de naissance) seul (= 6, 25)
m4_3live_v2__rotation__K0_6.25__seed1	lot F	rotation du crédit (décomposition et fermeture)	levier K_0 (capital de naissance) seul (= 6, 25)
m4_3live_v2__rotation__K0_6.25__seed2	lot F	rotation du crédit (décomposition et fermeture)	levier K_0 (capital de naissance) seul (= 6, 25)
m4_3live_v2__rotation__base__seed0	lot F	rotation du crédit (décomposition et fermeture)	régime de référence, avec le Gini du capital instrumenté
m4_3live_v2__rotation__base__seed1	lot F	rotation du crédit (décomposition et fermeture)	régime de référence, avec le Gini du capital instrumenté

run_id (simulation_lab)	groupe	apparaît dans	rôle / interventions
m4_3live_v2__rotation__base__seed2	lot F	rotation du crédit (décomposition et fermeture)	régime de référence, avec le Gini du capital instrumenté
m4_3live_v2__rotation__ delta_0.005__seed0	lot F	rotation du crédit (décomposition et fermeture)	levier \delta (dépréciation) seul (= 0, 005)
m4_3live_v2__rotation__ delta_0.005__seed1	lot F	rotation du crédit (décomposition et fermeture)	levier \delta (dépréciation) seul (= 0, 005)
m4_3live_v2__rotation__ delta_0.005__seed2	lot F	rotation du crédit (décomposition et fermeture)	levier \delta (dépréciation) seul (= 0, 005)
m4_3live_v2__rotation__delta_0.02__ seed0	lot F	rotation du crédit (décomposition et fermeture)	levier \delta (dépréciation) seul (= 0, 02)
m4_3live_v2__rotation__delta_0.02__ seed1	lot F	rotation du crédit (décomposition et fermeture)	levier \delta (dépréciation) seul (= 0, 02)
m4_3live_v2__rotation__delta_0.02__ seed2	lot F	rotation du crédit (décomposition et fermeture)	levier \delta (dépréciation) seul (= 0, 02)
m4_3live_v2__rotation__lam_15__ seed0	lot F	rotation du crédit (décomposition et fermeture)	levier \lambda (taux de naissance) seul (= 15)
m4_3live_v2__rotation__lam_15__ seed1	lot F	rotation du crédit (décomposition et fermeture)	levier \lambda (taux de naissance) seul (= 15)
m4_3live_v2__rotation__lam_15__ seed2	lot F	rotation du crédit (décomposition et fermeture)	levier \lambda (taux de naissance) seul (= 15)
m4_3live_v2__rotation__lam_60__ seed0	lot F	rotation du crédit (décomposition et fermeture)	levier \lambda (taux de naissance) seul (= 60)
m4_3live_v2__rotation__lam_60__ seed1	lot F	rotation du crédit (décomposition et fermeture)	levier \lambda (taux de naissance) seul (= 60)
m4_3live_v2__rotation__lam_60__ seed2	lot F	rotation du crédit (décomposition et fermeture)	levier \lambda (taux de naissance) seul (= 60)
m4_3live_v2__rotation__rho_0.5__ seed0	lot F	rotation du crédit (décomposition et fermeture)	levier \rho (taux d'appariement) seul (= 0, 5)
m4_3live_v2__rotation__rho_0.5__ seed1	lot F	rotation du crédit (décomposition et fermeture)	levier \rho (taux d'appariement) seul (= 0, 5)
m4_3live_v2__rotation__rho_0.5__ seed2	lot F	rotation du crédit (décomposition et fermeture)	levier \rho (taux d'appariement) seul (= 0, 5)
m4_3live_v2__rotation__rho_2__seed0	lot F	rotation du crédit (décomposition et fermeture)	levier \rho (taux d'appariement) seul (= 2)
m4_3live_v2__rotation__rho_2__seed1	lot F	rotation du crédit (décomposition et fermeture)	levier \rho (taux d'appariement) seul (= 2)
m4_3live_v2__rotation__rho_2__seed2	lot F	rotation du crédit (décomposition et fermeture)	levier \rho (taux d'appariement) seul (= 2)
m4_3live_v2__rotation__ sigma_0.005__seed0	lot F	rotation du crédit (décomposition et fermeture)	levier \sigma (choc multiplicatif) seul (= 0, 005)
m4_3live_v2__rotation__ sigma_0.005__seed1	lot F	rotation du crédit (décomposition et fermeture)	levier \sigma (choc multiplicatif) seul (= 0, 005)
m4_3live_v2__rotation__ sigma_0.005__seed2	lot F	rotation du crédit (décomposition et fermeture)	levier \sigma (choc multiplicatif) seul (= 0, 005)
m4_3live_v2__rotation__sigma_0.02__ seed0	lot F	rotation du crédit (décomposition et fermeture)	levier \sigma (choc multiplicatif) seul (= 0, 02)
m4_3live_v2__rotation__sigma_0.02__ seed1	lot F	rotation du crédit (décomposition et fermeture)	levier \sigma (choc multiplicatif) seul (= 0, 02)

run_id (simulation_lab)	groupe	apparaît dans	rôle / interventions
m4_3live_v2__rotation__sigma_0.02__seed2	lot F	rotation du crédit (décomposition et fermeture)	levier σ (choc multiplicatif) seul (= 0,02)
m4_3__d1__baseline__seed0	référence M4.3	conception (parité bit à bit sur 8000 pas); lots A et B	run M4.3 stocké servant de référence de parité
m4_3__d1__baseline__seed1	référence M4.3	protocole (justification de t_0)	t_converge_int_in = 948, lu dans son analysis.json
m4_3__d1__baseline__seed2	référence M4.3	protocole (justification de t_0)	t_converge_int_in = 841, lu dans son analysis.json

Tous ces runs sont ouvrables dans `simulation_lab` (`python3 -m simulation_lab.cli gui`, page « Résultats », lignée `m4_3live_v2_credit_soc`), avec leur `series.csv`, leur `tech_series.csv`, leurs `tension.csv` et `tension_agg.csv`, leur journal d'interventions et une figure `figures/macro_overview.png`. Le tableau complet, avec les paramètres et le statut de chaque run, est dans `results/analysis/traceability.csv`.